

ui.matplotlib 全面详解

`ui.matplotlib` 是 NiceGUI 框架中深度集成 Matplotlib 的可视化组件，专为简化 Web 环境下 Matplotlib 图表的创建与管理设计。其核心优势在于通过上下文管理器自动处理图表渲染与更新，无需手动管理图表对象与 Web 适配，支持 Matplotlib 全量图表类型与样式定制，是 Python 开发者快速构建 Web 可视化界面的高效工具。以下从核心特性、使用场景、详细配置、高级技巧等维度展开全面解析。

一、核心概述

1. 本质与定位

- 封装 Matplotlib 的 `Figure` 对象，通过上下文管理器（`with` 语句）提供简洁的图表创建流程，离开上下文时自动完成图表渲染与页面更新。
- 支持 Matplotlib 所有原生图表类型（折线图、柱状图、热力图、3D 图表等），兼容 Matplotlib 完整的样式配置与 API。
- 与 NiceGUI 生态深度融合，支持组件样式定制、事件监听、可见性绑定等 Web 应用必备功能，无需额外编写前端代码。

2. 核心参数（初始化时常用）

参数名	类型	说明
<code>**kwargs</code>	关键字参数	传递给 <code>matplotlib.figure.Figure</code> 的初始化参数，如 <code>figsize</code> （图表尺寸）、 <code>dpi</code> （分辨率）、 <code>facecolor</code> （背景色）等。
<code>style</code>	<code>Style[Self]</code>	图表容器的 CSS 样式，支持 Tailwind CSS（如 <code>bg-gray-50</code> 、 <code>rounded-lg</code> ）。
<code>classes</code>	<code>Classes[Self]</code>	图表容器的 HTML 类名，用于自定义样式复用。
<code>visible</code>	<code>bool</code>	图表初始可见性（默认 <code>True</code> ），支持后续动态修改或绑定。

二、基础使用场景与示例

1. 基础静态图表（上下文管理器用法）

通过 `with ui.matplotlib()` 开启上下文，直接操作内部的 `Figure` 对象（通过 `.figure` 属性获取），无需手动创建 `Figure` 和 `Axes`，离开上下文后自动渲染图表。

示例：基础折线图

```
from nicegui import ui
import numpy as np

# 上下文管理器模式：传入 figsize 配置图表尺寸（单位：英寸）
with ui.matplotlib(figsize=(5, 3)).figure as fig:
    # 获取当前坐标轴（等价于 plt.gca()）
    ax = fig.gca()
    # 生成数据并绘制
```

```

x = np.linspace(0.0, 5.0, 100)
y = np.cos(2 * np.pi * x) * np.exp(-x) # 衰减余弦曲线
ax.plot(x, y, '-', color='#28738a', linewidth=2)
# 配置坐标轴标签与标题
ax.set_xlabel('X Axis', fontsize=10)
ax.set_ylabel('Y Axis', fontsize=10)
ax.set_title('Damped Cosine Wave', fontsize=12, pad=10)
# 调整布局（避免标签溢出）
fig.tight_layout()

ui.run()

```

2. 多子图布局 (Subplots)

通过 `fig.add_subplot()` 或 `plt.subplots()` 创建多子图，在上下文内统一配置多个子图的内容与样式，自动批量渲染。

示例：2x1 多子图

```

from nicegui import ui
import numpy as np

with ui.matplotlib(figsize=(6, 4)).figure as fig:
    # 子图 1: 折线图
    ax1 = fig.add_subplot(2, 1, 1)
    x = np.linspace(0, 10, 100)
    ax1.plot(x, np.sin(x), color='red', label='sin(x)')
    ax1.legend()
    ax1.set_title('Sin wave')

    # 子图 2: 柱状图
    ax2 = fig.add_subplot(2, 1, 2)
    categories = ['A', 'B', 'C', 'D']
    values = [30, 45, 25, 50]
    ax2.bar(categories, values, color=['#b687ac', '#28738a', '#a78f8f', '#f1c40f'])
    ax2.set_title('Bar Chart')

    # 自动调整子图间距
    fig.tight_layout()

ui.run()

```

3. 动态更新图表数据

通过保存 `ui.matplotlib` 组件引用，在上下文外修改 `Figure` 对象的数据源，调用 `update()` 方法触发图表刷新，适用于实时数据监控场景。

示例：点击按钮切换图表数据

```

from nicegui import ui
import numpy as np

```

```

# 1. 创建组件并保存引用（通过上下文管理器获取 figure）
matplotlib_comp = ui.matplotlib(figsize=(5, 3))
with matplotlib_comp.figure as fig:
    ax = fig.gca()
    x = np.linspace(0, 5, 100)
    # 初始绘制 sin(x)
    line, = ax.plot(x, np.sin(x), color='blue', label='sin(x)')
    ax.set_ylim(-1.5, 1.5)
    ax.legend()

# 2. 定义动态更新函数
def toggle_data():
    current_label = line.get_label()
    if current_label == 'sin(x)':
        # 切换为 cos(x)
        line.set_data(x, np.cos(x))
        line.set_label('cos(x)')
    else:
        # 切换回 sin(x)
        line.set_data(x, np.sin(x))
        line.set_label('sin(x)')
    # 更新图例与图表
    ax = matplotlib_comp.figure.gca()
    ax.legend()
    matplotlib_comp.update() # 触发页面图表刷新

# 添加控制按钮
ui.button('Toggle sin/cos', on_click=toggle_data)

ui.run()

```

4. 定制图表容器样式

通过 `style` 和 `classes` 参数为图表容器添加 CSS 样式（支持 Tailwind CSS），实现背景色、圆角、阴影、边距等美化效果，适配 Web 页面设计。

示例：带样式的柱状图

```

from nicegui import ui
import numpy as np

# 配置容器样式：灰色背景、圆角、阴影、内边距
with ui.matplotlib(
    figsize=(6, 4),
    style='bg-gray-50 rounded-xl shadow-md p-4' # Tailwind 样式
).figure as fig:
    ax = fig.gca()
    x = ['Jan', 'Feb', 'Mar', 'Apr']
    y = [120, 180, 150, 220]
    ax.bar(x, y, color='#28738a', alpha=0.8)
    ax.set_title('Monthly Sales', fontsize=14, fontweight='bold')
    ax.set_ylabel('Sales Amount')
    # 隐藏顶部和右侧坐标轴

```

```
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

ui.run()
```

5. 3D 图表渲染

直接在上下文内创建 Matplotlib 3D 图表，`ui.matplotlib` 自动适配 3D 渲染逻辑，无需额外配置，支持 3D 折线图、散点图、曲面图等。

示例：3D 散点图

```
from nicegui import ui
import numpy as np
from mpl_toolkits.mplot3d import Axes3D

with ui.matplotlib(figsize=(6, 5)).figure as fig:
    # 创建 3D 坐标轴
    ax = fig.add_subplot(111, projection='3d')
    # 生成随机 3D 数据
    n = 100
    x = np.random.rand(n) * 10
    y = np.random.rand(n) * 10
    z = np.random.rand(n) * 10
    # 绘制 3D 散点图
    ax.scatter(x, y, z, c=z, cmap='viridis', s=50, alpha=0.7)
    ax.set_xlabel('X Axis')
    ax.set_ylabel('Y Axis')
    ax.set_zlabel('Z Axis')
    ax.set_title('3D Scatter Plot')

ui.run()
```

6. 事件监听（点击 /hover 交互）

通过 `on` 方法绑定 Web 事件（如点击、鼠标悬浮），结合 Matplotlib 的坐标转换功能，实现图表交互逻辑（如标记点击位置、显示数据详情）。

示例：点击图表显示坐标

```
from nicegui import ui
import numpy as np

# 1. 创建图表组件与状态标签
matplotlib_comp = ui.matplotlib(figsize=(5, 3))
click_label = ui.label('Click the chart to view coordinates')

# 2. 初始化图表
with matplotlib_comp.figure as fig:
    ax = fig.gca()
    x = np.linspace(0, 10, 100)
    ax.plot(x, np.sin(x), color='#b687ac')
```

```

ax.set_xlabel('X')
ax.set_ylabel('Y')

# 3. 绑定点击事件
def on_chart_click(e):
    # e.args 包含鼠标屏幕坐标 (x, y)
    screen_x, screen_y = e.args['x'], e.args['y']
    # 将屏幕坐标转换为图表数据坐标
    ax = matplotlib_comp.figure.gca()
    data_x, data_y = ax.transData.inverted().transform([[screen_x, screen_y]])[0]
    # 在点击位置添加红色标记
    ax.scatter(data_x, data_y, color='red', s=80, zorder=5)
    # 更新标签与图表
    click_label.set_text(f'Clicked at (X: {data_x:.2f}, Y: {data_y:.2f})')
    matplotlib_comp.update()

# 绑定 "click" 事件（支持的事件类型同 web 标准）
matplotlib_comp.on('click', on_chart_click)

ui.run()

```

7. 可见性绑定（动态显示 / 隐藏）

通过 `bind_visibility` 方法将图表可见性与其他组件状态绑定（如开关、复选框），实现图表的动态显示与隐藏，适配复杂界面的交互逻辑。

示例：开关控制图表显示

```

from nicegui import ui
import numpy as np

# 创建开关组件
show_chart = ui.switch('Show chart', value=True)

# 创建图表并绑定可见性
with ui.matplotlib(figsize=(5, 3)) as matplotlib_comp:
    with matplotlib_comp.figure as fig:
        ax = fig.gca()
        x = np.linspace(0, 5, 100)
        ax.plot(x, np.exp(-x) * np.cos(2*np.pi*x))
        ax.set_title('Damped Oscillation')
    # 绑定可见性：图表可见性跟随开关状态
    matplotlib_comp.bind_visibility(show_chart, 'value')

ui.run()

```

三、核心属性与方法详解

1. 常用属性

属性名	类型	说明
<code>figure</code>	<code>matplotlib.figure.Figure</code>	只读属性，获取组件关联的 Matplotlib Figure 对象，用于修改图表数据、样式等。
<code>visible</code>	<code>BindableProperty</code>	可读可写，控制图表是否可见（支持绑定，如 <code>bind_visibility</code> ）。
<code>style</code>	<code>Style[Self]</code>	可读可写，图表容器的 CSS 样式，支持动态修改（如 <code>matplotlib_comp.style('bg-blue-50')</code> ）。
<code>classes</code>	<code>Classes[Self]</code>	可读可写，图表容器的 HTML 类名，用于样式复用。
<code>html_id</code>	<code>str</code>	图表在 HTML DOM 中的唯一 ID（v2.16.0+ 支持），用于自定义 JS 交互。

2. 核心方法

方法名	作用	参数说明
<code>update()</code>	手动触发图表刷新，修改 <code>figure</code> 后必须调用此方法才能在页面更新图表。	-
<code>on(type, handler)</code>	绑定 Web 事件（如 <code>click</code> 、 <code>mousemove</code> ），实现图表交互。	<code>type</code> ：事件类型（如 <code>click</code> ）； <code>handler</code> ：回调函数，接收事件参数 <code>e</code> （含屏幕坐标等）。
<code>bind_visibility(target)</code>	双向绑定图表可见性到目标对象的属性（如开关的 <code>value</code> 属性）。	<code>target_object</code> ：目标对象（如 <code>ui.switch()</code> ）； <code>target_name</code> ：属性名（默认 <code>visible</code> ）。
<code>bind_visibility_from(target)</code>	单向绑定可见性（从目标对象到图表）。	参数同 <code>bind_visibility</code> ，仅单向同步。
<code>bind_visibility_to(target)</code>	单向绑定可见性（从图表到目标对象）。	参数同 <code>bind_visibility</code> ，仅单向同步。
<code>set_visibility(visible)</code>	直接设置图表可见性（布尔值）。	<code>visible</code> ： <code>True</code> （显示）/ <code>False</code> （隐藏）。
<code>tooltip(text)</code>	为图表添加悬浮提示框。	<code>text</code> ：提示文本内容。
<code>delete()</code>	删除图表组件及关联的 Matplotlib Figure 对象，释放内存。	-

四、高级技巧与注意事项

1. 性能优化

- 上下文复用：避免频繁创建 `ui.matplotlib` 组件，动态更新场景优先复用现有 `figure` 对象（修改数据而非重建图表）。
- 布局优化：使用 `fig.tight_layout()` 或 `fig.subplots_adjust()` 避免标签溢出，减少图表重绘次数。
- 分辨率控制：通过初始化参数 `dpi` 调整图表分辨率（默认 100），Web 显示建议 100-150，打印场景可提升至 300。

2. 样式定制技巧

- 图表内部样式：通过 Matplotlib 的 `ax.set_*` 方法（如 `set_title`、`set_xlabel`）定制坐标轴、图例、标题样式，支持字体、颜色、大小等配置。
- 容器样式：结合 Tailwind CSS 快速实现响应式布局（如 `style='width: 100%; max-width: 800px'`），适配不同屏幕尺寸。
- 主题复用：使用 Matplotlib 内置主题（如 `plt.style.use('seaborn-v0_8')`）或自定义样式表，统一图表风格。

3. 动态更新最佳实践

- 避免重复创建 `Axes`：动态更新时，优先修改现有 `Line2D`、`BarContainer` 等对象的数据（如 `line.set_data()`），而非每次调用 `ax.plot()` 重建。
- 批量更新：多次修改图表后集中调用 `update()`，减少页面刷新次数，提升性能。
- 内存管理：动态更新频繁的场景，定期清理无用数据（如删除过期标记点），避免 `figure` 对象过大导致内存泄漏。

4. 兼容性与版本说明

- 依赖版本：要求 Matplotlib $\geq 3.0.0$ ，NiceGUI $\geq 2.0.0$ （`html_id` 需 v2.16.0+）。
- 3D 图表支持：需安装 `mpl_toolkits.mplot3d`（Matplotlib 自带），无需额外依赖。
- 事件兼容性：`on` 方法支持的事件为 Web 标准事件（如 `click`、`mousemove`），不支持 Matplotlib 原生交互事件（如 `button_press_event`），需通过屏幕坐标转换实现类似功能。

5. 调试技巧

- 查看 `figure` 配置：打印 `matplotlib_comp.figure` 可获取当前图表的完整配置，排查样式或数据问题。
- 事件参数调试：在回调函数中打印 `e.args`，查看事件的完整参数（如鼠标坐标、组件 ID 等）。
- 图表不显示排查：检查是否遗漏 `update()` 方法、`figure` 是否正确初始化、容器样式是否设置了 `display: none`。

五、与 ui.pyplot 的核心区别

NiceGUI 中 `ui.matplotlib` 与 `ui.pyplot` 均为 Matplotlib 集成组件，但设计理念与使用场景存在差异，选择时可参考下表：

特性	ui.matplotlib	ui.pyplot
核心设计	基于 <code>Figure</code> 对象，上下文管理器优先	基于 <code>pyplot</code> 接口，兼容 Matplotlib 脚本风格
初始化方式	上下文内直接操作 <code>figure</code> ，自动渲染	需手动创建 <code>Figure</code> 并传入组件
动态更新	修改 <code>figure</code> 后调用 <code>update()</code>	相同逻辑，无本质差异
易用性	简洁，无需手动管理 <code>Figure</code> 生命周期	灵活，适配已有 Matplotlib 代码
适用场景	新开发的 Web 可视化项目，追求简洁性	复用现有 Matplotlib 脚本，需兼容旧代码

六、总结

`ui.matplotlib` 是 NiceGUI 中面向 Web 场景优化的 Matplotlib 集成组件，核心优势在于：

- 1. 简洁高效：通过上下文管理器自动处理图表渲染与生命周期，减少模板代码。
- 2. 深度集成：与 NiceGUI 生态无缝衔接，支持样式定制、事件监听、可见性绑定等 Web 核心功能。
- 3. 全量兼容：支持 Matplotlib 所有图表类型与样式配置，无需学习新的可视化语法。
- 4. 低门槛：Python 开发者无需前端知识，即可快速构建交互式 Web 图表。

适用于数据报表、实时监控面板、科学计算可视化、Web 数据分析工具等场景，尤其适合新开发项目或追求代码简洁性的场景。